

Hashing

"Hash map వాడు, అది $O(1)$ " – ఇది DSA లో అత్యధికంగా ఇచ్చే సలహా, మరియు అత్యల్పంగా పరీక్షించబడినది.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది – $n = 10$ లక్షల దగ్గర **Set, sorting** కంటే **2.5 రెట్లు నెమ్మది**, మరియు bitmap కంటే 10 రెట్లు. రెండేది – "దాదాపు సరైన" key **99.3% సరిగ్గా** పనిచేసి **737 తప్పు జతలు** ఇస్తుంది. మూడేది – `{}` వాడితే "మొదటి ఏకైక అక్షరం" **54.4% cases లో తప్పు**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo `run` చేసినదే – timings తో సహా, మరియు \$6 lo ఈ machine lo ఉన్న **88,758 నిజమైన పదాల** నిఘంటువుతో.

HASHING యొక్క నిజమైన ప్రశ్న "ఏ నిర్మాణం" కాదు – "ఏ KEY"

`Map`, `Set`, `{}` – ఇవి పనిముట్లు. ఆలోచన ఇది: మీరు ఒక సమానత్వాన్ని ఒక string గా మార్చుతున్నారు.

"ఈ రెండు పదాలు anagrams" → అదే key.

"ఈ రెండు బిందువులు ఒకే గడిలో ఉన్నాయి" → అదే key.

ఆ మార్పిడి నష్టం లేనిది (lossless) అయితే algorithm సరైనది. కొంచెం నష్టం ఉన్నా – \$6 lo కొలుస్తాం – అది **99.3% సరిగ్గా** పనిచేసి మిగతా చోట్ల మౌనంగా తప్పు.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Hashing ఏమి కొంటుంది – మరియు ఏ ధరకి
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: $O(1)$ అంటే వేగం కాదు

3. Step – ఒకే ప్రశ్న, నాలుగు విధానాలు
4. మొదటి విరుపు – Set, sorting కంటే నెమ్మది

Part 3 – రెండో విరుపు: key నష్టం

5. Step – key ఒక సమానత్వాన్ని సూచిస్తుంది
6. రెండో విరుపు – 99.3% సరైన key
7. Step – canonical key ఎలా ఎంచుకోవాలి

Part 4 – మూడో విరుపు: JavaScript యొక్క రెండు ఉచ్చులు

8. Step – `{}` vs `Map` vs `Set`
9. మూడో విరుపు – 54.4% తప్పు, ఒక్క అక్షరం మార్చకుండా
10. Step – ఎప్పుడు ఏది వాడాలి

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code · 24,000 cases · mutation testing
12. బతికిన మార్పు – ఒక విధానం, ఒక bug కాదు
13. Interview lo ఎలా రాయాలి

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Hashing ఏమి కొంటుంది – మరియు ఏ ధరకి

Hashing ఒక్కటే పని చేస్తుంది: "ఇది ఇంతకుముందు చూశానా?" అనే ప్రశ్నకి, ఎన్ని వస్తువులు ఉన్నా, **అదే సమయంలో** జవాబు ఇస్తుంది.

అది $O(n^2)$ ని $O(n)$ చేస్తుంది. కానీ "అదే సమయం" అనేది **స్థిరమైనది, చిన్నది కాదు** – మరియు ఆ తేడా యే §4.

"నకలు ఉందా?" • అన్ని విలువలు *వేరువేరు* (ఏదీ ముందే ఆగదు)

n	brute $O(n^2)$	sort $O(n \log n)$	Set $O(n)$	bitmap $O(n)$
100	4 us	1 us	1 us	1 us
1,000	300 us	6 us	14 us	4 us
10,000	30.6 ms	74 us	266 us	37 us
1,00,000	(నెమ్మది)	1.3 ms	3.1 ms	373 us
10,00,000	(నెమ్మది)	15.5 ms	39.0 ms	3.9 ms

BRUTE FORCE నిజంగా చెడ్డది – అది ఈ DOC LO వివాదం కాదు

$n = 10,000$ దగ్గర **30.6 ms**. $n = 1$ లక్ష దగ్గర అది గంటలవుతుంది.

కాబట్టి "ఏదైనా $O(n)$ పద్ధతి కావాలి" అనేది సరైన ముగింపు.

వివాదం ఏమిటంటే – ఆ $O(n)$ పద్ధతి hash set కావాలా అనేది. కుడివైపు మూడు నిలువు వరుసలు చూడండి.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Keys పూర్ణ సంఖ్యలా , మరియు పరిధి దట్టమా?	అవును అంటే hash అవసరం లేదు – ఒక array చాలు, మరియు అది 10 రెట్లు వేగం (§4)
n ఎంత?	చిన్నది అయితే brute force లేదా sort వేగం (§4)

ప్రశ్న	ఎందుకు అడుగుతున్నాం
"ఒకటే" అంటే ఏమిటి?	ఇదే key యొక్క నిర్వచనం – §6
Key ఒక array లేదా object కావచ్చా?	అవును అంటే §9 – JS lo అది మౌనంగా విఫలం
క్రమం ముఖ్యమా?	అవును అంటే {} వాడకూడదు – §9 lo 54.4% తప్పు
Memory పరిమితి ఉందా?	Hash set ఒక్కో మూలకానికి ~50 bytes; bitmap ఒక్కో దానికి 1 bit

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా కొలిచారని తెలుస్తుంది

"Keys పూర్ణ సంఖ్యలా, మరియు వాటి పరిధి తెలుసా?"

ఎందుకంటే జవాబు "అవును, 0 నుంచి 10 లక్షల వరకు" అయితే – **hash map అవసరమే లేదు**. ఒక

Uint8Array చాలు, మరియు §4 lo అది **10 రెట్లు వేగం** మరియు memory lo చాలా చిన్నది.

"Hash map వాడు" అనేది ఒక reflex. ఈ ప్రశ్న అడగడం అంటే మరు reflex కి బదులు **data ని** చూస్తున్నారని అర్థం.

Part 2 – మొదటి విరుపు: $O(1)$ అంటే వేగం కాదు

3. Step – ఒకే ప్రశ్న, నాలుగు విధానాలు

```
const brute = a => { for (let i=0;i<a.length;i++) for (let j=i+1;j<a.length;j++)
    if (a[i]===a[j]) return true; return false; };
const sortWay = a => { const s = Int32Array.from(a).sort();
    for (let i=1;i<s.length;i++) if (s[i]===s[i-1]) return true;
    return false; };
const setWay = a => { const s = new Set();
    for (const x of a) { if (s.has(x)) return true; s.add(x); }
    return false; };
const bitmap = (a,M) => { const b = new Uint8Array(M);
    for (const x of a) { if (b[x]) return true; b[x]=1; } return
    false; };
```

Big-O ప్రకారం క్రమం స్పష్టం: **bitmap = Set ($O(n)$) < sort ($O(n \log n)$) < brute ($O(n^2)$).**

4. మొదటి విరుపు – Set, sorting కంటే నెమ్మది

n	sort $O(n \log n)$	Set $O(n)$	bitmap $O(n)$
10,000	74 us	266 us	37 us
1,00,000	1.3 ms	3.1 ms	373 us
10,00,000	15.5 ms	39.0 ms	3.9 ms

N = 10 లక్షల దగ్గర: SORT 15.5 MS, SET 39.0 MS – SET 2.5 రెట్లు నెమ్మది

Big-O తప్పు చెప్పలేదు. $O(n)$ నిజంగా $O(n \log n)$ కంటే మెరుగు – పెద్ద n దగ్గర, మరియు స్థిరాంకాలు సమానంగా ఉంటే.

ఇక్కడ అవి సమానం కాదు. ఒక్కో Set.add కి: hash లెక్కించడం, bucket వెతకడం, memory కేటాయించడం, pointer వెంబడి వెళ్ళడం. ఒక్కో sort పోలిక: రెండు పక్కపక్క సంఖ్యలు.

$\log_2(10 \text{ లక్షలు}) = 20$. అంటే sort ఒక్కో మూలకానికి ~20 చొక్కైన పనులు చేస్తుంది. Set ఒక్కో మూలకానికి 1 ఖరీదైన పని. $20 \times \text{చౌక} < 1 \times \text{ఖరీదు}$.

మరియు BITMAP 3.9 MS – SET కంటే 10 రెట్లు వేగం

Keys పూర్ణ సంఖ్యలు మరియు పరిధి తెలిస్తే, "hash" అనే భాగం మొత్తం **అనవసరం**. $b[x] = 1$ అనేది ఒక్క memory రాత.

మరియు memory: 10 లక్షల మూలకాలకి Uint8Array **1 MB**. ఒక Set అదే data కి **50 MB** పైనే.

చిన్న n దగ్గర ఇంకా వేరే కథ:

చిన్న n దగ్గర ఏది వేగం? (అన్నీ వేరువేరు, 2 లక్షల సార్లు)

n	brute	sort	Set	bitmap	విశేష
4	8 ns	62 ns	36 ns	21 ns	brute
8	28 ns	70 ns	93 ns	29 ns	brute
16	90 ns	95 ns	192 ns	53 ns	bitmap
32	335 ns	239 ns	403 ns	114 ns	bitmap
64	1334 ns	352 ns	768 ns	232 ns	bitmap
128	5452 ns	775 ns	1500 ns	585 ns	bitmap

ఆరు వరుసల్లో ఏ ఒక్కదాంట్లోనూ SET గెలవలేదు

$n = 4$ దగ్గర brute force **8 ns**, Set **36 ns** – 4.5 రెట్లు తేడా, మరియు brute force "చెడ్డ" algorithm.

ఇది Set చెడ్డదని కాదు. ఇది – **Set యొక్క స్థిరాంకం పెద్దది**, మరియు n చిన్నగా ఉన్నప్పుడు స్థిరాంకమే మొత్తం.

ఆచరణాత్మక నియమం: Set ని దాని వేగం కోసం కాదు, దాని **సాధారణత** కోసం ఎంచుకోండి – keys ఏ రకమైనవైనా, పరిధి తెలియకపోయినా, n అపరిమితం అయినా అది పనిచేస్తుంది. Keys దట్టమైన పూర్ణ సంఖ్యలు అయితే array వాడండి.

Part 3 – రెండో విరుపు: key నష్టం

5. Step – key ఒక సమానత్వాన్ని సూచిస్తుంది

Anagrams ని గుంపులుగా చేయడం. "ఒకటే" అంటే: **అవే అక్షరాలు, అదే సంఖ్యలో.**

ఆ సమానత్వాన్ని ఒక string గా మార్చాలి. నాలుగు సహజమైన ఆలోచనలు:

```
w => w.split('').sort().join('') // sorted అక్షరాలు
w => String(w.split('').reduce((s,c)=>s+c.charCodeAt(0),0)) // ASCII మొత్తం
w => { let p=1n; for (const c of w) p*=BigInt(c.charCodeAt(0)); return String(p); }
// ASCII లబ్ధం
w => w.length + w[0] // పొడవు + మొదటి అక్షరం
```

నాలుగు **anagrams** కి **ఒకే key** ఇస్తాయి. ప్రశ్న వేరే: **anagrams కానివాటికి కూడా ఒకే key ఇస్తాయా?**

6. రెండో విరుపు – 99.3% సరైన key

ఈ machine lo ఉన్న నిజమైన నిఘంటువు – **88,758 పదాలు** (≤ 8 అక్షరాలు):

key ఎంపిక	గుంపులు	తప్పు జతలు (anagrams కానివి కలిసిపోయినవి)
sorted అక్షరాలు (సరైనది)	73,056	0
అక్షరాల ASCII మొత్తం	643	1,51,88,217 ఉదా: aal, aid, beg
అక్షరాల ASCII లబ్ధం	72,580	737 ఉదా: abducent, ethician
పొడవు + మొదటి అక్షరం	206	5,54,08,725 ఉదా: aa, ab, ad

రెండు మరియు నాలుగే వరుసలు స్పష్టంగా తప్పు – మరియు అదే వాటిని సురక్షితంగా చేస్తుంది

ASCII మొత్తం: **643 గుంపులు** మాత్రమే (నిజం 73,056). aal, aid, beg ఒకే గుంపులో. మీరు దీన్ని ఒక్క ఉదాహరణ నడిపితే వెంటనే కనిపెడతారు.

పొడవు + మొదటి అక్షరం: **206 గుంపులు**. ఇంకా స్పష్టం.

పెద్దగా తప్పు అయితే అది త్వరగా బయటపడుతుంది. అదే మంచి విషయం.

కానీ మూడో వరుస - **ASCII** లబ్ధం - ఇదే ప్రమాదకరమైనది

72,580 గుంపులు, నిజం 73,056. తేడా **0.65%**. మీరు ఒక dashboard lo ఈ రెండు సంఖ్యలు చూస్తే ఏమీ అనిపించదు.

మరియు **737 తప్పు జతలు** అక్కడ ఉన్నాయి:

abducent · ethician – రెండింటి ASCII లబ్ధం **14190278350248000**.

sorted: abcdentu | acehiint – స్పష్టంగా anagrams కాదు.

లబ్ధం ఒక **దాదాపు-injective** function. "దాదాపు" అంటే పరీక్షల్లో దొరకదు, **production lo** దొరుకుతుంది.

7. Step – canonical key ఎలా ఎంచుకోవాలి

ఒక **KEY** సరైనదా అని తేల్చే ఒక్క ప్రశ్న

"ఈ key నుంచి నేను అసలు వస్తువు యొక్క సమానత్వ తరగతిని తిరిగి నిర్మించగలనా?"

`sorted('listen') = 'eilnst'` → దీని నుంచి "ఏ అక్షరాలు, ఎన్ని" అనేది **పూర్తిగా** తెలుస్తుంది. ✓

`sum('listen') = 649` → దీని నుంచి అక్షరాలు తెలియవు. **సమాచారం పోయింది.** ✗

`product('listen')` → దీని నుంచి సిద్ధాంతపరంగా అక్షరాలు తీయొచ్చు (ప్రధాన కారణంక విభజన)... కానీ ASCII సంఖ్యలు ప్రధాన సంఖ్యలు కావు, కాబట్టి $2 \times 110 = 4 \times 55$ లాంటి ఢీకొట్టుడు సాధ్యం. ✗

ఆచరణలో వాడే canonical keys:

సమానత్వం	Key
అవే అక్షరాలు (anagram)	<code>w.split('').sort().join('')</code> – లేదా 26 లెక్కల array
అదే గడి (grid cell)	<code>`\${row},\${col}`</code> – విభాజకంతో (§11)
అదే వరుస/నిలువు/పెట్టి (sudoku)	<code>`r\${r}-\${d}`</code> , <code>`c\${c}-\${d}`</code> , <code>`b\${b}-\${d}`</code>
అదే sorted tuple	<code>[...t].sort().join(',')</code>
అదే సంఖ్యల సమితి	sorted + join, Set కాదు (§9)

26-లెక్కల ARRAY ఎందుకు మెరుగు కావచ్చు

`sort()` ఒక్కో పదానికి $O(L \log L)$. బదులుగా 26 అక్షరాల లెక్కలని ఒక string గా రాస్తే $O(L)$:

```
const key = w => { const c = new Array(26).fill(0);  
  for (const ch of w) c[ch.charCodeAt(0)-97]++; return c.join(','); };
```

ఇది కూడా **నష్టం లేనిది** – మరియు విభాజకం , ఉంది, కాబట్టి `[1,11]` మరియు `[11,1]` ఢీకొట్టువు. ఆ విభాజకం ఎందుకు అవసరమో §11 lo mutation testing చూపిస్తుంది.

Part 4 – మూడో విరుపు: JavaScript యొక్క రెండు ఉచ్చులు

8. Step – {} vs Map vs Set

JavaScript lo మూడూ "hash map" లాగా కనిపిస్తాయి. అవి కావు.

1. Array లేదా object ని key గా వాడితే

5 జతలు, అందులో వేరువేరు 2
new Set(pairs).size = 5 ← ఏదీ dedupe కాలేదు
string key తో = 2 ← సరైనది

కారణం: [1,2] !== [1,2]. రెండు వేర్వేరు objects.

SET REFERENCE ప్రకారం పోలుస్తుంది, విషయం ప్రకారం కాదు

new Set([[1,2],[3,4],[1,2]]) యొక్క పరిమాణం 3, 2 కాదు.

మరియు ఇది crash కాదు, హెచ్చరిక కాదు – మీ dedupe ఏమీ చేయకుండా విజయవంతంగా ముగుస్తుంది.

పరిష్కారం: key ని ఒక string గా మార్చడం (§7 యొక్క పట్టిక).

రెండో ఉచ్చు ఇంకా సూక్ష్మమైనది:

2. JS object lo keys క్రమం

చేర్చిన క్రమం : 10, 2, banana, 1, apple
Object.keys() : 1, 2, 10, banana, apple
Map keys : 10, 2, banana, 1, apple ← చేర్చిన క్రమమే

పూర్ణ-సంఖ్య-లాంటి keys పెరుగుతున్న క్రమంలో ముందుకి వస్తాయి – మీరు ఏ క్రమంలో చేర్చినా. ఇది JavaScript యొక్క నిర్వచనంలో ఉన్న ప్రవర్తన, ఒక bug కాదు.

9. మూడో విరుపు – 54.4% తప్పు, ఒక్క అక్షరం మార్చకుండా

"మొదటిసారి పునరావృతం కాని అక్షరం" – ఒక classic problem. రెండు అమలులు, ఒకే తర్కం:

```
// {} తో  
const c = {}; for (const ch of s) c[ch] = (c[ch]||0)+1;
```

```
for (const k of Object.keys(c)) if (c[k]===1) return k;

// Map తో
const c = new Map(); for (const ch of s) c.set(ch, (c.get(ch)||0)+1);
for (const [k,v] of c) if (v===1) return k;
```

2,00,000 యాదృచ్ఛిక strings (అంకెలు + అక్షరాలు)

ఏ నిర్మాణం	తప్పు జవాబులు	%
{} object	1,08,769	54.4%
Map	0	0.0%

ఉదా: "a1" → object 1, నిజం a

"A1" – రెండు అక్షరాలు · జవాబు A, వచ్చినది 1

రెండూ ఏకైకమే. కాబట్టి "మొదటి" అనేది క్రమం మీద ఆధారపడుతుంది.

{a:1, '1':1} lo Object.keys() ఇస్తుంది ['1','a'] – ఎందుకంటే '1' పూర్ణ-సంఖ్య-లాంటిది, అది ముందుకి దూకుతుంది.

Code lo ఏ తప్పు లేదు. తర్కం సరైనది. నిర్మాణం వేరే ఒప్పందం ఇచ్చింది.

మరియు ఇది అంకెలు ఉన్నప్పుడే – అందుకే ఇది దాక్కుంటుంది

Input lo అక్షరాలు మాత్రమే ఉంటే {} మరియు Map ఒకే జవాబు ఇస్తాయి. LeetCode యొక్క చాలా test cases అక్షరాలే.

కానీ వాడుకరి ids, product codes, sudoku అంకెలు, IP addresses – వీటిలో అంకెలు ఉంటాయి, మరియు అప్పుడు **54.4%**.

10. Step – ఎప్పుడు ఏది వాడాలి

పరిస్థితి	ఏమి వాడాలి	ఎందుకు
Keys దట్టమైన పూర్ణ సంఖ్యలు	Uint8Array / Int32Array	§4 – 10 రెట్లు వేగం , 50 రెట్లు తక్కువ memory
క్రమం ముఖ్యం	Map	§9 – {} పూర్ణ-సంఖ్య keys ని తిరగేస్తుంది
Keys strings, క్రమం అనవసరం	Map లేదా {}	రెండూ సరే – కానీ Map అలవాటు చేసుకోవడం సురక్షితం
Key ఒక tuple/array	arr.join(విభాజకం)	§8 – reference పోలిక
n చాలా చిన్నది (< 16)	ఒక array + linear scan	§4 – Set యొక్క స్థిరాంకం పెద్దది

ఒక సరళమైన అలవాటు

JavaScript lo hash map కావాలంటే Map రాయండి, {} కాదు.

`{}` ఒక *object* – దానికి *prototype* ఉంది (`obj['toString']` ఒక function ఇస్తుంది!), keys strings గా మారతాయి, మరియు పూర్ణ-సంఖ్య keys క్రమం మారుతుంది.

`Map` ఒక *hash map* – ఏ రకమైన key అయినా, చేర్చిన క్రమం, *prototype* లేదు, మరియు `.size` ఉంది.

ఒకే ఒక్క తేడా ఉంది: JSON కి {} కావాలి. అప్పుడు చివర్లో మార్పిడి, మధ్యలో కాదు.

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code • 24,000 cases • mutation testing

ఎనిమిది నియమాలు, అన్నీ brute force తో పోల్చినవి: ఒక గుంపులోని ప్రతి జత నిజంగా anagrams, anagrams అయిన ప్రతి జత ఒకే గుంపులో, గుంపులు input ని పూర్తిగా విభజించాలి, firstUnique = input క్రమంలో మొదటి ఏకైక అక్షరం, dedupe తర్వాత సరిగ్గా వేరువేరు tuples, dedupe తర్వాత నకళ్ళు ఉండకూడదు, జత ఉంటే twoSum తప్పక కనుగొనాలి, twoSum ఇచ్చిన indices వేరువేరు మరియు నిజంగా కలిపితే target.

రెండో నియమం ముఖ్యమైనది: మొదటిది "తప్పుగా కలపలేదు" అని మాత్రమే చెబుతుంది. రెండోది "తప్పుగా విడదీయలేదు" అని చెబుతుంది – మరియు రెండూ కలిపితేనే key సరైనదని రుజువు.

400 runs • 16,000 పదాలు • 24,000 strings • 24,000 twoSums
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

words	16,000
groups	8,057
strings	24,000
tuples	12,000
twoSums	24,000
found	5,538

==== mutation testing ====

మార్పు లేని code	→ 0/400 విఫలం
anagram key = అక్షరాల ASCII మొత్తం	→ 150/150 విఫలం
ఉదా: "abcb" మరియు "ccaa" anagrams కాదు, ఒకే గుంపులో ఉన్నాయి	
anagram key = పొడవు + మొదటి అక్షరం	→ 150/150 విఫలం
ఉదా: "abc" మరియు "acc" anagrams కాదు, ఒకే గుంపులో ఉన్నాయి	
firstUnique lo Map కి బదులు {}	→ 150/150 విఫలం
ఉదా: firstUnique("1a10") = 0, అశించినది a	
dedupe lo string key కి బదులు array నే	→ 57/150 విఫలం
ఉదా: dedupe 30, వేరువేరు 29	
tupleKey విభజకం లేకుండా (join(""))	→ 8/150 విఫలం
ఉదా: dedupe 29, వేరువేరు 30	
twoSum lo *మొదటి* index కి బదులు *చివరి* index	→ 0/150 విఫలం
twoSum lo తనిఖీకి ముందే seen lo చేర్చితే	→ 150/150 విఫలం
ఉదా: twoSum([1,4], 2) = [0,0] – ఒకే index రెండుసార్లు	

విభాజకం మార్పు మొదట 0/150 చూపించింది - మరియు అది నా DATA యొక్క లోపం

`tupleKey` lo `join('\u0000')` ని `join('')` గా మార్చాను. నా fuzz lo tuples విలువలు 0-3 మాత్రమే - ఒక్క అంకె. అప్పుడు `[1,2].join('') = "12"`, మరియు ఢీకొట్టడానికి దారే లేదు.

విలువలని 0-29 కి పెంచాక: `[1,23]` మరియు `[12,3]` రెండూ "123". ఇప్పుడు 8/150.

DSA Deep 01 యొక్క అదే పాఠం: ఒక mutation బతికితే, మీ code కంటే ముందు మీ data ని అనుమానించండి.

12. బతికిన మార్పు - ఒక విధానం, ఒక bug కాదు

ఒక్క మార్పు నిజంగా బతికింది: `twoSum` lo విలువ యొక్క మొదటి index ఉంచడానికి బదులు చివరి index ఉంచడం.

```
if (!seen.has(nums[i])) seen.set(nums[i], i); // మొదటిది ఉంచుతుంది
seen.set(nums[i], i); // చివరిది ఉంచుతుంది ← mutation
```

దాన్ని విడిగా పరీక్షించాను:

5,00,000 cases
రెండూ చెల్లుబాటు కాని సార్లు : 0
జవాబు వేరుగా వచ్చిన సార్లు : 15506 (3.10%)
ఉదా: `a=[7,4,6,4,7,1,5]` `t=12` → మొదటిది `[0,6]`, చివరిది `[4,6]`

రెండూ ఎప్పుడూ చెల్లుబాటు అయ్యే జవాబు ఇస్తాయి - 3.10% సార్లు వేరే జవాబు

`[0,6]` → `7 + 5 = 12` ✓. `[4,6]` → `7 + 5 = 12` ✓. రెండూ సరైనవే.

అంటే ఈ పంక్తి ఒక bug నివారణ కాదు - అది ఒక tie-breaking విధానం. "ఏదైనా ఒక జత ఇవ్వ" అనే ప్రశ్నకి రెండూ సరైనవి.

LeetCode "exactly one solution" అని హామీ ఇస్తుంది, కాబట్టి అక్కడ తేడా లేదు. కానీ ప్రశ్న "మొదటి జత" అని అడిగితే - అప్పుడు ఈ పంక్తి భారం మోస్తుంది.

కాబట్టి ఇది బతికిన నాలుగో రకం: పరీక్ష బలహీనం కాదు, code చచ్చినది కాదు - ప్రశ్న ఆ తేడాని అడగడం లేదు.

13. Interview lo ఎలా రాయాలి

అడుగు	ఏమి చేయాలి
1	"ఒకటే" అంటే ఏమిటో ఒక వాక్యంలో చెప్పండి. అదే మీ key
2	ఆ key నష్టం లేనిదా అని అడగండి – "దీని నుంచి సమానత్వ తరగతిని తిరిగి నిర్మించగలనా?"
3	Keys దట్టమైన పూర్ణ సంఖ్యలా అని చూడండి – అయితే array , Map కాదు
4	JS lo ఎప్పుడూ Map , {} కాదు
5	Key ఒక tuple అయితే విభాజకంతో join చేయండి

14. Follow-ups – collisions, dense ranges, rolling hash

"Hash collisions నిజంగా ఎంత ముఖ్యం?"

భాష యొక్క hash map lo – దాదాపు ఎప్పుడూ కాదు, ఎందుకంటే అవి తమ hash ని **యాదృచ్ఛికంగా** ఎంచుకుంటాయి (process కి ఒకసారి), కాబట్టి ఒక దాడి చేసేవాడు ముందుగా డీకొట్టే keys తయారు చేయలేడు. మీరు **మీ సొంత** hash రాస్తే – అప్పుడు ముఖ్యం, మరియు అదే LLD Deep 08 (HashMap) యొక్క విషయం. ఈ doc lo నేను కొలిచిన డీకొట్టుడు **hash యొక్క డీకొట్టుడు కాదు** – అది **key యొక్క డీకొట్టుడు** (§6), మరియు అది పూర్తిగా మీ చేతిలో ఉంటుంది.

"పూర్ణ సంఖ్యల పరిధి పెద్దది కానీ విలువలు చాలా తక్కువ ఉంటే?"

అప్పుడు bitmap పనికిరాదు (LLD Deep 19 lo 10,000 keys కి **447 GB** అని కొలిచాను). రెండు ఎంపికలు: **Map** (సాధారణ జవాబు), లేదా విలువలు దాదాపు వరుసగా ఉంటే వాటిని **కుదించడం** (coordinate compression) – sort చేసి 0..k-1 కి మ్యాప్ చేయడం, ఆపై array వాడటం.

"Substring ని key గా వాడాలంటే ప్రతిసారీ string తయారు చేయాలా?"

అది $O(L)$ ప్రతి substring కి. **Rolling hash** (Rabin-Karp) ఒక substring నుంచి తర్వాతి దానికి hash ని **$O(1)$** lo నవీకరిస్తుంది. ఖర్చు: ఇది ఒక **ఉజ్జాయింపు** – రెండు వేర్వేరు substrings కి ఒకే hash రావచ్చు, కాబట్టి ఒక hit దొరికాక **నిజంగా పోల్చి చూడాలి**. నేను దీన్ని ఈ doc lo కొలవలేదు.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **rolling hash** (పైన), **Map యొక్క resize ప్రవర్తన** (Set timings lo అది ఉంది, కానీ విడిగా కొలవలేదు), మరియు **coordinate compression** యొక్క crossover – ఎప్పుడు అది Map కంటే మెరుగే నేను కొలవలేదు.

15. ఏమి నేర్చుకున్నాం

1. $O(1)$ అంటే వేగం కాదు – అంటే n తో పెరగదు. $n = 10$ లక్షల దగ్గర `Set` sorting కంటే **2.5 రెట్లు** నెమ్మది, `bitmap` కంటే **10 రెట్లు**.

2. చిన్న n దగ్గర `Set` అరు పరిమాణాల్లో ఒక్కసారి కూడా గెలవలేదు. $n = 4$ దగ్గర brute force **4.5 రెట్లు** వేగం.

3. నిజమైన ప్రశ్న "ఏ నిర్మాణం" కాదు, "ఏ key". Key మీ సమానత్వాన్ని సరిగ్గా సూచించాలి.

4. పెద్దగా తప్పు key సురక్షితం; కొంచెం తప్పు key ప్రమాదకరం. ASCII మొత్తం **643 గుంపులు** (వెంటనే కనిపిస్తుంది). ASCII లబ్ధం **72,580 vs 73,056** – 0.65% తేడా, **737 తప్పు జతలు**.

5. `Set` విషయం ప్రకారం పోల్చు. `new Set([[1,2],[1,2]]).size === 2`, మరియు అది మౌనంగా విజయవంతం అవుతుంది.

6. `{}` ఒక hash map కాదు. పూర్ణ-సంఖ్య-లాంటి keys క్రమం మారుతుంది → "మొదటి ఏకైక అక్షరం" **54.4% తప్పు**.

7. Tuple key కి విభాజకం తప్పనిసరి. లేకపోతే `[1, 23]` మరియు `[12, 3]` ఒకటే – మరియు నా fuzz అది ఒక అంకె విలువలతో పట్టుకోలేకపోయింది.

8. బతికిన mutation కి నాలుగో అర్థం ఉంది. బలహీన పరీక్ష, చచ్చిన code, harness పరిమితి – లేదా ప్రశ్న ఆ తేడాని అడగకపోవడం (§12).

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Hashing అంటే వెతకడం కాదు – అది ఒక సమానత్వాన్ని ఒక పేరుగా మార్చడం.

ఆ పేరు రెండు షరతులు పాటించాలి: **ఒకటే వస్తువులకి ఒకే పేరు**, మరియు **వేరువేరు వస్తువులకి వేరే పేరు**. మొదటిది సులభం – చాలా keys అది చేస్తాయి. రెండోది ఎవరూ పరీక్షించరు.

మరియు అది విరిగినప్పుడు crash కాదు. §6 lo అది **99.3% సరైనది**. §9 lo అది **అక్షరాలు మాత్రమే ఉన్న inputs మీద ఖచ్చితం**.

కాబట్టి key ఎంచుకున్న ప్రతిసారీ ఒక్క ప్రశ్న అడగండి: "దీని నుంచి నేను అసలు దాన్ని తిరిగి నిర్మించగలనా?"